

AI-Powered Accounting & Finance Assistant

A Research Paper on AI Automation in Accounting Workflows

PydanticAI	FastAPI	Next.js	Supabase	Gemini 2.0 Flash	Python uv
------------	---------	---------	----------	------------------	-----------

Prepared By	Muhammad Talha Khan
Organization	Cyber Nuts
Submission Date	July 2026
Assignment	Full-Stack AI Developer Intern — Phase 1
Primary Model	Google Gemini 2.0 Flash
Framework	PydanticAI + FastAPI + Next.js + Supabase

CONFIDENTIAL — Internship Submission Document — Cyber Nuts 2026

Table of Contents

1.	Introduction	
2.	Core Responsibilities of an Accountant / CA	
	2.1 Daily Tasks	
	2.2 Monthly Tasks	
	2.3 Yearly & Ongoing Tasks	
3.	AI Automation Feasibility Mapping	
4.	Agentic AI Frameworks — Comparative Analysis	
	4.1 OpenAI Agents SDK	
	4.2 LangGraph	
	4.3 CrewAI	
	4.4 PydanticAI / Custom Tool-Calling Loop (Our Choice)	
5.	AI Model Selection	
6.	System Architecture	
7.	Derived Feature List	
8.	Conclusion	
9.	References	

A Research Paper on AI Automation in Accounting Workflows

The accounting and finance domain is one of the most structured, rule-driven sectors in modern business — a characteristic that makes it exceptionally well-suited for AI-driven automation. This paper systematically analyzes the comprehensive role of an accountant or Chartered Accountant (CA), maps each responsibility to its automation potential using contemporary AI technologies, compares leading agentic AI frameworks for financial applications, justifies our technology and model selection, and presents the final architecture for a full-stack AI-powered accounting assistant. The outcome is an implementation-ready design grounded in both accounting theory and modern software engineering practice.

1 Introduction

2 Core Responsibilities of an Accountant / CA

2.1 Daily Tasks

2.2 Monthly Tasks

2.3 Yearly & Ongoing Tasks

3 AI Automation Feasibility Mapping

4 Agentic AI Frameworks — Comparative Analysis

4.1 OpenAI Agents SDK

4.2 LangGraph

4.3 CrewAI

4.4 PydanticAI / Custom Tool-Calling Loop

5 AI Model Selection

6 System Architecture for the AI Accounting Assistant

7 Derived Feature List

8 Conclusion

9 References

1. Introduction

Accounting is the backbone of every business, large or small. From recording daily transactions and managing petty cash to filing statutory returns and preparing audited balance sheets, an accountant's responsibilities

span a broad spectrum of data-intensive, rule-governed, and judgment-driven tasks. Traditionally, these functions have required years of specialized training and immense manual effort. However, the emergence of Large Language Models (LLMs) and agentic AI frameworks presents a transformative opportunity: to augment accounting professionals with AI systems that can understand natural language, execute deterministic financial computations, detect anomalies, and generate insightful reports — all while maintaining a verifiable audit trail.

This paper is structured as Phase 1 of a larger software development effort. Its primary goal is to establish a rigorous research foundation before a single line of application code is written. We adopt the Spec-Driven Development (SDD) methodology, in which thorough research and formal specifications precede implementation, ensuring that architectural decisions are evidence-based rather than ad hoc.

The central research questions this paper addresses are:

- What does a professional accountant or CA actually do on a daily, monthly, and yearly basis?
- Which of these tasks are best automated by AI, and which must remain human-supervised?
- Which agentic framework and LLM model combination is most appropriate for a high-accuracy, auditable financial AI assistant?
- What architecture should this full-stack application adopt?

2. Core Responsibilities of an Accountant / CA

To design an AI system that genuinely automates accounting workflows, we must first understand those workflows comprehensively. Below is a structured breakdown of accountant responsibilities by frequency.

2.1 Daily Tasks

#	Task	Description
1	Journal Entry Recording	Recording every financial transaction (purchases, sales, receipts, payments) into the general ledger using double-entry bookkeeping principles (debit/credit).
2	Cash & Bank Book Maintenance	Updating the cashbook and passbook with all inflows and outflows; reconciling minor discrepancies in real time.
3	Petty Cash Management	Tracking small day-to-day office expenditures (stationery, tea, transport) and ensuring they are properly categorized.
4	Invoice Processing	Receiving, verifying, and entering vendor invoices; generating and dispatching sales invoices to clients.

#	Task	Description
5	Accounts Payable / Receivable Monitoring	Tracking amounts owed by the business (AP) and amounts owed to the business (AR); flagging overdue entries.
6	Daily Expense Categorization	Classifying each expenditure into the correct account head (rent, utilities, salaries, marketing, etc.) for accurate reporting.
7	Banking Transactions	Recording bank transfers, EFTs, cheque clearances, and online payments; matching with bank statements.

2.2 Monthly Tasks

#	Task	Description
1	Bank Reconciliation Statement (BRS)	Matching every entry in the company's cashbook against the bank statement to identify discrepancies, uncleared cheques, or errors.
2	Trial Balance Preparation	Extracting all ledger account balances and ensuring total debits equal total credits — a fundamental accuracy check.
3	Profit & Loss (P&L) Statement	Summarizing all revenues and expenses over the month to compute net profit or net loss.
4	Fixed & Recurring Expense Review	Reviewing monthly obligations such as rent, salaries, SaaS subscriptions, utility bills, loan EMIs, and insurance premiums for changes or anomalies.
5	GST / VAT / Sales Tax Computation	Calculating tax liabilities on sales and purchases; reconciling input and output tax credits; preparing returns.
6	Payroll Processing	Calculating gross and net salaries, deducting statutory contributions (PF, ESSI, PAYE), and disbursing payroll.
7	Departmental Cost Analysis	Breaking down expenses by department, project, or cost center to identify areas of overspending.
8	Accounts Aging Analysis	Reviewing receivables and payables by age buckets (0–30, 31–60, 61–90+ days) to prioritize collections and payments.

#	Task	Description
9	Monthly Audit & Anomaly Review	Scanning all entries for duplicate postings, unusual amounts, incorrect categories, and authorization policy violations.

2.3 Yearly & Ongoing Tasks

#	Task	Description
1	Balance Sheet Preparation	Presenting the full financial position of the business — assets, liabilities, and equity — as of a specific date (usually fiscal year end).
2	Cash Flow Statement	Analyzing the sources and uses of cash across operating, investing, and financing activities.
3	Statutory Audit Support	Preparing and providing documentation (ledgers, vouchers, reconciliation statements) to external auditors.
4	Income Tax Filing	Computing taxable income, applicable deductions, and filing annual tax returns in compliance with the jurisdiction's tax code.
5	Fixed Asset Register Maintenance	Tracking all fixed assets (computers, machinery, furniture), recording acquisition cost, depreciation, and disposal.
6	Budget Preparation & Variance Analysis	Setting the annual financial budget and, during the year, comparing actual performance against budget to identify variances.
7	Internal Controls Review	Periodically reviewing financial policies, authorization matrices, and internal procedures to prevent fraud and error.
8	Financial Forecasting	Using historical data to project future revenues, expenses, and cash flows to support business planning.

3. AI Automation Feasibility Mapping

Not all accounting tasks are equally amenable to AI automation. The table below classifies each task by its automation potential and the AI technique best suited for it.

Task	Automation Potential	Primary AI Technique	Human Oversight Required
Journal Entry Recording	High	NL Parsing → Structured DB Write	Validation only
Invoice Processing	High	OCR + NLP Extraction + DB Write	Fraud-check spot review
Expense Categorization	High	Classification via LLM / Rule Engine	Low
P&L; Generation	High	Deterministic SQL Aggregation	None
Balance Sheet Generation	High	Deterministic SQL Aggregation	None
Bank Reconciliation	High	Matching Algorithm + Anomaly Flags	Exception review
Monthly Audit	High	Statistical Outlier Detection + LLM Summary	Human reviews flagged items
Trial Balance	High	Automated DB Calculation	Spot review
GST/Tax Computation	Medium	Rule-Based Engine (jurisdiction-specific)	CPA verification
Payroll Processing	Medium	Automated Calculation + Audit Trail	HR sign-off
Budget Variance Analysis	Medium	Data Aggregation + LLM Narrative	Finance manager review
Financial Forecasting	Medium	Time-Series ML / LLM with historical data	Analyst oversight
Statutory Audit Support	Low	Document retrieval (RAG)	Full CPA/Auditor control
Internal Controls Review	Low	Policy Q&A; via RAG	Full management control

Key Insight: The most impactful automation opportunity lies in the high-frequency, data-entry-intensive tasks (journal entries, expense categorization, P&L;, balance sheet, audit flagging). These are where AI provides the greatest ROI with minimal risk.

Critical Design Principle: For financial AI, accuracy is non-negotiable. The AI agent must NEVER hallucinate financial figures. All monetary computations must be executed as deterministic SQL queries against the database, with the LLM serving exclusively as a natural-language interface and an explanation engine — not as a calculator.

4. Agentic AI Frameworks — Comparative Analysis

Selecting the right agentic framework is one of the most consequential architectural decisions for an AI accounting assistant. An "agentic framework" orchestrates how an LLM reasons, selects tools, executes actions, handles errors, and produces a final response. We evaluated four leading frameworks.

4.1 OpenAI Agents SDK

Overview: Officially released by OpenAI in 2025, the Agents SDK provides a structured way to define agents with system prompts, tools (functions), and handoffs between agents. It is purpose-built for production agentic workflows.

Strengths:

- Native integration with OpenAI's GPT-4o family, which has best-in-class tool calling (function calling) accuracy.
- Clean abstractions for tool definition using Python type hints.
- Built-in support for parallel tool calls, tracing, and guardrails.
- Excellent documentation and enterprise support.

Weaknesses:

- Vendor lock-in to OpenAI's API.
- Less flexible for complex multi-agent graphs compared to LangGraph.

Suitability for Financial AI: High. Its strict tool-calling discipline aligns well with our requirement that all financial figures come from deterministic SQL tools, not from LLM inference.

4.2 LangGraph

Overview: An extension of LangChain developed by LangChain Inc., LangGraph models the agent's reasoning as a directed acyclic graph (DAG) or cyclic graph, allowing stateful, multi-step workflows with explicit conditional routing.

Strengths:

- Highly flexible — can model complex, multi-step financial workflows (e.g., invoice → categorize → reconcile → flag → summarize).
- Excellent support for "human in the loop" patterns.
- Provider-agnostic: works with OpenAI, Anthropic, Google, and local models.
- Native streaming and persistence via LangGraph Cloud.

Weaknesses:

- Steep learning curve; requires understanding of graph theory concepts.
- Can be over-engineered for simpler agentic tasks.
- Performance overhead from graph state management.

Suitability for Financial AI: High for complex multi-agent pipelines. However, introduces unnecessary complexity for our current scope.

4.3 CrewAI

Overview: CrewAI is a high-level, role-based multi-agent framework where multiple AI "agents" with distinct roles collaborate to complete tasks. For example, a "Data Entry Agent" and a "Financial Report Agent" could work together.

Strengths:

- Intuitive role-based model that mirrors organizational team structures.
- Good for decomposing complex tasks across multiple specialized agents.
- Active community and growing ecosystem.

Weaknesses:

- Abstraction can obscure what each agent is actually doing, making debugging difficult.
- Less predictable tool call behavior compared to OpenAI Agents SDK.
- Overhead of inter-agent communication for simple tasks.

Suitability for Financial AI: Medium. The role-based model is conceptually appealing but adds orchestration complexity that doesn't provide proportional benefit for our MVP scope.

4.4 PydanticAI / Custom Tool-Calling Loop (Our Choice)

Overview: PydanticAI is a Python agent framework from the creators of Pydantic, designed for production AI applications with a focus on type safety, structured outputs, and testability. Alternatively, a custom tool-calling loop built on top of the provider SDK (Anthropic, OpenAI, or Google) can provide maximum control.

Strengths:

- **First-class Pydantic integration:** Since our backend already uses Pydantic for all schema validation, PydanticAI provides a seamless bridge between the AI agent and our data models.
- **Type-safe tool definitions:** Tools are defined with full Python type hints, and Pydantic validates all tool call arguments and return values automatically.
- **Provider-agnostic:** Supports OpenAI, Anthropic (Claude), Google (Gemini), and local Ollama models out of the box, with easy switching via environment variables — eliminating vendor lock-in.
- **Designed for testing:** Built-in support for mocking LLM calls in unit tests, crucial for testing financial accuracy.
- **Transparent and predictable:** Minimal abstraction layers mean we always know exactly what the agent is doing — critical for financial auditability.
- **Excellent async support:** Fully async by design, compatible with FastAPI's async architecture.

Weaknesses:

- Younger framework compared to LangGraph; fewer third-party integrations.
- Smaller community than LangChain/CrewAI.

Suitability for Financial AI: Highest. PydanticAI's emphasis on type safety, Pydantic-first design, testability, and provider-agnosticism makes it the ideal choice for our full-stack accounting application built with FastAPI.

Framework Comparison Summary

Criterion	OpenAI Agents SDK	LangGraph	CrewAI	PydanticAI ■
Type Safety	Medium	Low	Low	High
Pydantic Integration	Medium	Medium	Low	Native
Provider Agnostic	No	Yes	Yes	Yes
Testability	Medium	Medium	Low	High
Complexity	Low	High	Medium	Low
Auditability	Medium	High	Low	High
FastAPI Compatibility	Good	Good	Good	Excellent
Vendor Lock-in Risk	High	None	None	None

Decision: PydanticAI is selected as our agentic framework for its unmatched combination of type safety, testability, Pydantic-first design, and provider agnosticism.

5. AI Model Selection

Our AI agent is model-agnostic by design (via environment variable configuration), but our primary recommendation is:

Primary: Google Gemini 2.0 Flash

Reasons:

- **Excellent tool-calling accuracy:** Gemini 2.0 Flash demonstrates precise, JSON-schema-compliant function calling — critical for our financial tools.
- **Speed:** Flash-tier models are significantly faster than full models, providing sub-2-second response times for typical accounting queries.
- **Cost efficiency:** Generous free tier and competitive pricing for internship/development environments.
- **Long context window:** 1M token context window allows entire transaction histories to be analyzed in a single call if needed.
- **Google Cloud integration:** Natural integration with Supabase (PostgreSQL on Google Cloud infrastructure).

Secondary: Anthropic Claude 3.5 Sonnet

Reasons:

- Best-in-class tool use reliability and instruction following.
- Consistently accurate structured outputs (JSON).
- Excellent for financial narrative generation (P&L; summaries, audit explanations).
- Higher cost but justified for production accuracy requirements.

Tertiary: OpenAI GPT-4o Mini

Reasons:

- Excellent cost-performance ratio.
- Reliable JSON mode and function calling.
- Widely documented and well-tested.

All three models will be supported via our .env configuration, with Gemini as the default.

Model Selection Criterion Matrix

Criterion	Gemini 2.0 Flash	Claude 3.5 Sonnet	GPT-4o Mini
Tool-Calling Accuracy	★★★★■	★★★★★	★★★★■
Latency	★★★★★	★★★██	★★★★■
Cost	★★★★★	★★███	★★★★■
Context Window	1M tokens	200K tokens	128K tokens
Free Tier	Yes	No	Limited
Our Choice	■ Primary	Secondary	Tertiary

6. System Architecture for the AI Accounting Assistant

6.1 High-Level Architecture

The system adopts a **three-tier architecture** enhanced with an AI agent layer:

```

+-----+
|          PRESENTATION TIER          |
|  Next.js 14 + TypeScript (App Router)  |
|  Dark Mode | Glassmorphism | Micro-Animations | Charts |
|  AI Chat Drawer | Dashboard | Reports |
+-----+
|          | HTTPS REST API (JSON)          |
+-----+
|          LOGIC TIER                    |
|  FastAPI (Python, managed by uv)      |
|  +-----+                             |
|  |          AI Agent Layer              |
|  |  PydanticAI Agent + Tool Definitions |
|  |  -> create_transaction  -> list_transactions |
|  |  -> generate_pnl        -> generate_balance_sheet |
|  |  -> run_monthly_audit   -> summarize_spending |
|  +-----+                             |
|  +-----+                             |
|  |          REST API Endpoints          |
|  |  /api/v1/transactions  /api/v1/reports |
|  |  /api/v1/audit         /api/v1/agent/chat |
|  +-----+                             |
|  Pydantic Schemas | SQLAlchemy Async ORM | Alembic |
+-----+
|          | Async PostgreSQL (asyncpg)          |
+-----+
|          DATA TIER                    |
|  Supabase (PostgreSQL as a Service)    |
|  Tables: accounts | categories | transactions | audit_logs |
|  Row Level Security (RLS) | Real-time subscriptions |
|  Supabase Storage (future: invoice document uploads) |
+-----+
|          |
+-----+
|          EXTERNAL AI SERVICES          |
|  Google Gemini API (primary) | Anthropic Claude (secondary) |
|  Provider-agnostic via PydanticAI + .env configuration |
+-----+

```

6.2 Database Schema Overview

Core Tables (PostgreSQL / Supabase):

- **accounts** — Chart of accounts (Cash, Bank, Revenue, Expenses, Assets, Liabilities, Equity)
- **categories** — Expense/income categories (Rent, Utilities, Salaries, Sales, etc.)
- **transactions** — Every financial entry (date, amount, type, category_id, account_id, description, created_by)
- **audit_logs** — Immutable record of all changes to transactions (who, what, when)

Referential Integrity: All foreign keys enforced at the database level. All monetary values stored as `DECIMAL(15, 2)` to avoid floating-point precision errors.

6.3 AI Agent Interaction Pattern

The AI agent operates in a strict tool-calling loop:

```

User Natural Language Input
↓
PydanticAI Agent (LLM)
↓ (selects appropriate tool)
Tool Function (Python, calls DB via SQLAlchemy)
↓ (returns structured Pydantic model)
PydanticAI Agent (LLM)
↓ (formats user-facing response)
Natural Language Response + Structured Data

```

Zero-Hallucination Guarantee: All numbers returned to the user come exclusively from verified database queries. The LLM is only used for (a) parsing intent, (b) selecting the correct tool, (c) formatting human-readable output.

6.4 Security Considerations

- All database credentials stored exclusively in environment variables (never in code).
- Supabase Row Level Security (RLS) policies enforce data access boundaries.
- API endpoint authentication via JWT tokens (Supabase Auth integration).
- All monetary inputs validated by Pydantic schemas (type, range, format) before any DB write.
- Audit logs are append-only — no update or delete operations permitted on `audit_logs`.

7. Derived Feature List

Based on the research above, the following features constitute our Phase 3 implementation scope:

7.1 Core Financial Features

- 1 **Transaction Entry (CRUD):** Add, view, edit, and delete financial transactions with full categorization and account mapping.
- 2 **Daily Expense Logging:** Dedicated quick-entry form for petty cash and daily expenses.
- 3 **Recurring Monthly Expenses:** Track fixed obligations (rent, salaries, subscriptions) with monthly auto-recognition.
- 4 **General Ledger View:** Full, filterable record of all transactions by date, category, account, and type.

7.2 Financial Reporting

- 5 **Profit & Loss (P&L;) Statement:** Auto-generated monthly P&L; summarizing all revenue and expense accounts.
- 6 **Balance Sheet:** Auto-generated snapshot of assets, liabilities, and equity as of any selected date.

7

Spending Summary by Category: Visual breakdown of expenditure by category for any time period.

8

Accounts Aging Report: Breakdown of receivables and payables by age bucket.

7.3 AI-Powered Features

9

Natural Language Entry: Add transactions, query ledgers, and request reports via plain English / Urdu (e.g., "Add July rent expense of 50,000").

10

AI-Generated Financial Summaries: The AI writes plain-English explanations of P&L; and balance sheet results.

11

Monthly Anomaly Detection: AI scans all transactions and flags statistical outliers, duplicates, and policy violations.

12

Interactive Audit Review: Flagged items are presented in a human-review UI with accept/reject actions.

7.4 System & UX Features

13

Dark Mode Dashboard: Premium visual design with glassmorphism cards, vibrant accent colors, and smooth animations.

14

Responsive Layout: Fully functional on desktop and tablet screens.

15

Export Reports: Download P&L; and Balance Sheet as PDF or CSV.

16

Audit Trail Viewer: Full history of all changes with timestamps.

8. Conclusion

This research has established a comprehensive foundation for the AI-Powered Accounting & Finance Assistant. We have documented the full scope of professional accounting responsibilities, assessed their AI automation potential, rigorously compared leading agentic frameworks, and selected PydanticAI as our implementation framework based on its superior type safety, testability, and Pydantic-first design that aligns perfectly with our FastAPI backend. Gemini 2.0 Flash is selected as the primary LLM for its accuracy, speed, generous free tier, and cloud infrastructure alignment with our Supabase (PostgreSQL) database.

The system architecture enforces a zero-hallucination guarantee by mandating that all financial figures originate from deterministic SQL queries, with the LLM serving exclusively as a natural-language interface. This architecture is both technically sound and practically valuable — it provides the auditability required by accounting standards while delivering the conversational UX that modern users expect.

With this research foundation, the project will advance to Phase 2 (Spec-Driven Design) and Phase 3 (Full-Stack Implementation), each guided by the decisions documented here.

9. References

- 1 International Federation of Accountants (IFAC). *Handbook of International Quality Management, Auditing, Review, Other Assurance, and Related Services Pronouncements*. 2023 Edition.
- 2 Pydantic AI Documentation. *PydanticAI — Agent Framework for Production AI Applications*. <https://ai.pydantic.dev/>, 2025.
- 3 Anthropic. *Claude 3.5 Sonnet Model Card & Tool Use Documentation*. <https://docs.anthropic.com>, 2025.
- 4 Google DeepMind. *Gemini 2.0 Flash Technical Report*. <https://deepmind.google/technologies/gemini/>, 2025.
- 5 LangChain. *LangGraph Documentation — Stateful, Multi-Actor Applications with LLMs*. <https://langchain-ai.github.io/langgraph/>, 2025.
- 6 CrewAI Inc. *CrewAI Framework Documentation*. <https://docs.crewai.com>, 2025.
- 7 OpenAI. *OpenAI Agents SDK Documentation*. <https://openai.github.io/openai-agents-python/>, 2025.
- 8 Supabase. *Supabase Architecture Documentation — PostgreSQL as a Service*. <https://supabase.com/docs>, 2025.
- 9 FastAPI. *FastAPI Documentation — Modern, Fast Web Framework for Python*. <https://fastapi.tiangolo.com>, 2025.
- 10 Institute of Chartered Accountants of Pakistan (ICAP). *Syllabus and Competency Framework for Chartered Accountants*. 2023.